Understanding observability metrics: Types, golden signals, and best practices

- Observability metrics provide insights into the performance, behavior, and health of applications, systems, and infrastructure.
- Observability enables teams to understand a system's internal state by analysing telemetry data such as metrics, logs, and traces.
- observability refers to collecting and analysing telemetry data, logs, metrics, and traces from a variety of sources to gain detailed visibility into application behavior.
- When implemented effectively, observability improves issue detection, accelerates troubleshooting, enhances performance, and supports data driven decision making.
- Observability is a practice that gives organizations a 360-degree view of their environments and operations.

3 pillars of observability:

Observability is traditionally built on three core pillars: metrics, logs, and traces.

1. **Metrics**

    - Metrics are raw numerical data points collected from hardware, software, and websites.
    - Metrics are used for monitoring resource usage, performance, and user behavior.
    - metrics tell monitoring and observability teams **what** is happening in their systems.

Core types of observability metrics

i. **Application metric**

    - These metrics monitor application performance and availability.
    - Examples: Response time, throughput, request rates, and error counts.
    - These metrics allow engineers to monitor application performance and availability.

ii. **System metrics (infrastructure metrics)**

- These metrics reflects health of hardware, cloud resources, and operating systems.
- Examples: CPU utilization, disk I/O, network throughput, memory usage, instance uptime, container resource utilization, and service availability.
- These metrics provide insights into the performance of cloud resources, virtual machines, containers, and other underlying components.

iii. Business metrics

- These metrics connect technical performance to business outcomes.
- Examples: conversion rates, average transaction value, and user retention rate.

2. Logs

- Logs are timestamped event records generated by systems, applications, infrastructure and networks.
- They provide event details and context, allowing engineers to understand why issues occur.
- Logs are recorded in structured and unstructured formats, which represents a storage challenge.

Types of Logs

i. System Logs

- These include events like connection attempts, errors, and configuration changes.

ii. Application logs

- They record software changes, CRUD operations, application authentication, and other events to help diagnose issues.

iii. Network logs

- Network logs record data from events that take place on a network or device, including network traffic, security events, and user activity.

3. **Traces**

- Traces are telemetry signals that let engineers see applications and services from a user-session perspective.
- Traces tell DevOps teams where issues are occurring in their environments.
- They are the foundation of proactive monitoring.
- By analyzing traces, engineers can discover which metrics or logs are related to a particular issue.
- Traces are particularly useful for **API calls**, **Server-to-server communication**, **Front-end to back-end interactions**, etc.

4. **Profiles (Emerging Fourth Pillar)**

- Profiling analyzes stack traces and code-level performance.
- Profiling provides deep visibility into application behavior at the code level.
- It helps detect Memory allocation issues, Inefficient data structures and CPU bottlenecks.

The 4 golden signals for SRE teams

1. **Latency**

- Measures the time taken to process a request. High latency negatively impacts user experience.

2. **Traffic**

- Represents the volume of requests handled by a system.
- It helps understand user demand and scaling requirements.

3. **Errors**

- Tracks the rate of failed requests or operations. Monitoring error patterns helps prevent recurring issues.

4. **Saturation**

- Indicates how close a system is to its capacity limits (CPU, memory, etc.).
- It helps prevent overloads.

These four golden signals are key to effective observability practices as they provide insights into the health and performance of IT systems.

Best Practices for Implementing Observability Metrics

1. **Define Clear Objectives**
   - Identify what matters most to your organization. Avoid collecting unnecessary data.
2. **Use Open Standards**
   - Adopt vendor-neutral frameworks such as OpenTelemetry to avoid vendor lock-in.
3. **Leverage Automation**
   - Automate data collection, monitoring, and alerting to improve efficiency and response time.
4. **Customize Dashboards**
   - Tailor dashboards to match organizational goals instead of relying solely on default visualizations.

Observability metrics with Elastic

- Provides a **unified platform** for collecting, monitoring, and analyzing observability metrics.
- Supports **end-to-end visibility** across the entire technology stack.
- Enables collection, storage, and visualization of metrics from **any data source**.
- Integrates with the **Search AI Platform** to accelerate problem detection and resolution.